

Train the agent, not just the model

Sergio Paniego Blanco ([@sergiopaniego](#))
Machine Learning Engineer @ Hugging Face



Quick intro to Hugging Face 🤗

AI open platform for building, sharing and deploying AI (HF Hub):

- Models, datasets, apps...
- Posts, blogs, courses, papers, leaderboards...

2.8M

public models

950K+

public datasets

The screenshot displays the Hugging Face web interface. At the top, there's a navigation bar with links for Models, Datasets, Spaces, Community, Docs, Enterprise, and Pricing. Below this, the main content area is divided into three columns. The left column contains a sidebar with user profile links (sergiopaniago), organization links (Hugging Face, The LLM Course, etc.), and resource links (Getting Started, Documentation, etc.). The middle column shows a 'Following' feed with updates from users like Steveeeeeen and Ethank01. The right column features a 'Trending' section with popular models such as A11baba-NLP/Tongyi-DeepResearch-30B-A3B and ibn-granite/granite-docling-258M. The bottom of the page shows more updates from the community.

1M+

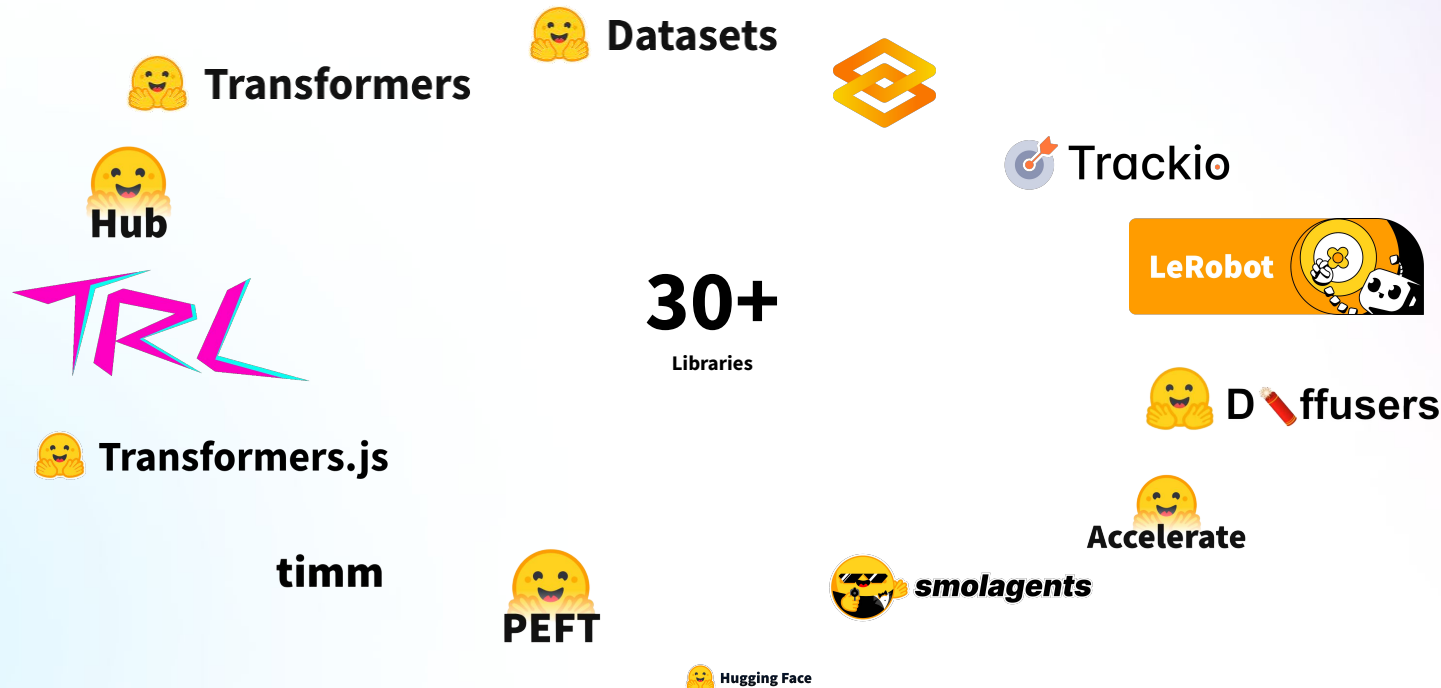
daily downloads

1M+

daily visitors

Quick intro to Hugging Face

Set of open source projects to empower AI development and advancement



How a workflow looks like?

Find model → **Customize** → Deploy



Transformers



OpenEnv



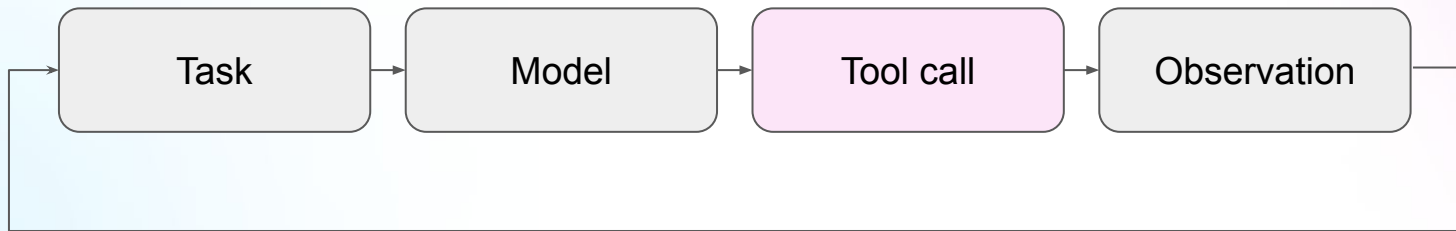
Coding agents

Pi, Hermes Agent, Claude Code, Codex... running on models (Claude, GPT, GLM...)

Tools for coding: *read a file, edit code, run tests...*

Agent = model + harness (model in a loop)

- Harness → execution layer
- Scaffolding → behavior layer



Glossary in case you feel lost about AI agent terminology: <http://hf.co/blog/agent-glossary>

Why training your own coding agent?

- **Specialized:** a small model tuned for a specific task can match a big general one on it
- **Cheaper:** the small model is far cheaper to run at scale
- **Yours:** your weights, run anywhere, private, no vendor lock-in...
- **Understood:** so we understand how coding agents actually work!

John Schulman (co-creator of ChatGPT) agrees on post-training :) <https://x.com/johnschulman2/status/2072132258463719658>

So, how do you build a model like that?

How is a model trained? Training stages

Pre-training: teaches general capacities such as language use, broad reasoning, and world knowledge

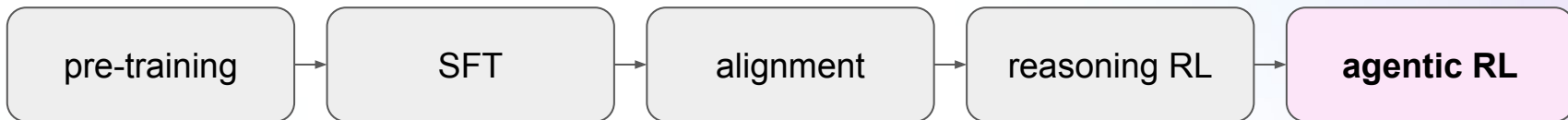
Mid-training: domain knowledge (code, medical, databases, internal docs...)

Post-training (TRL + OpenEnv):

- Behavior: instruction following, alignment, reasoning
- Agentic skills: RL envs

TRL and OpenEnv operate in post-training

A recipe that keeps evolving



The training recipe has evolved over the years:

- 2022-2023: SFT → reward model → PPO (InstructGPT, ChatGPT)
- 2024: SFT → DPO/PPO → RLVR (verifiable rewards) (Tülu 3, OLMo 3)
- 2025: RLVR becomes the main driver (DeepSeek-R1)
- 2026: Agentic RL → on-policy distillation (GLM-5 has “Agentic RL” stage)

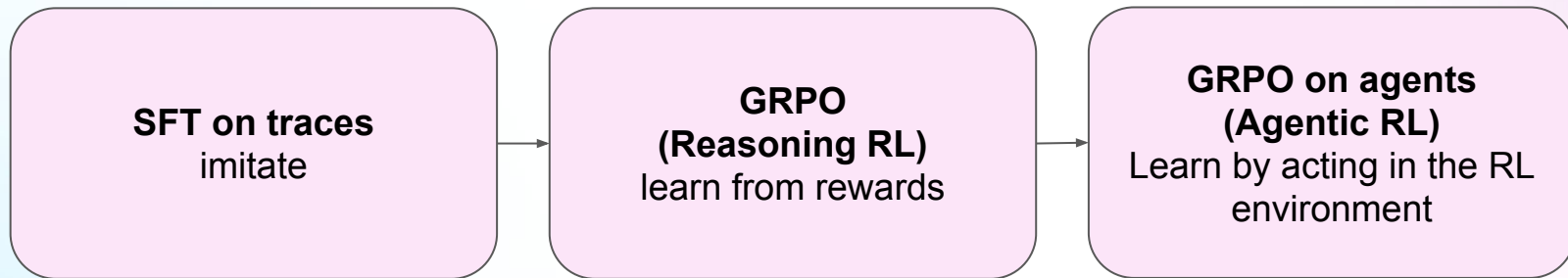
Timeline via Nathan Lambert’s recipe review <https://www.interconnects.ai/p/frontier-post-training-recipe-review>

Zoom into post-training for coding agents

Three methods take us to our coding agent

The model that we use already chats and follows instructions (it's not a base model)

All of these can be done using TRL + OpenEnv + open models



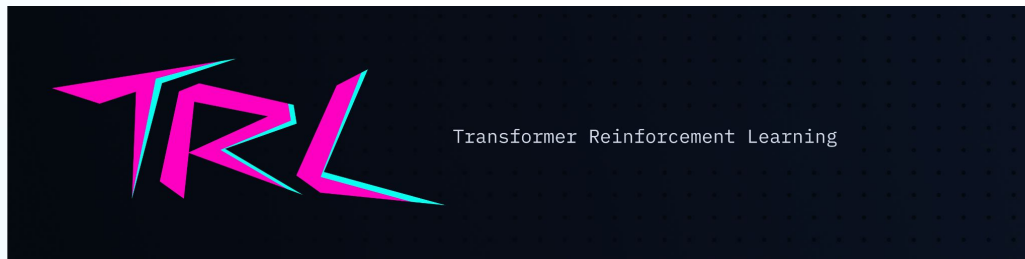
TRL: Fine-tuning, alignment & reasoning

HF's post-training library, built on top of the transformers Trainer.

Ready-to-use specialized trainers, no reimplementing from scratch

- **Online** (generate and learn in real time): GRPO, AsyncGRPO, PPO...
- **Offline** (learn from static data): SFT, DPO...

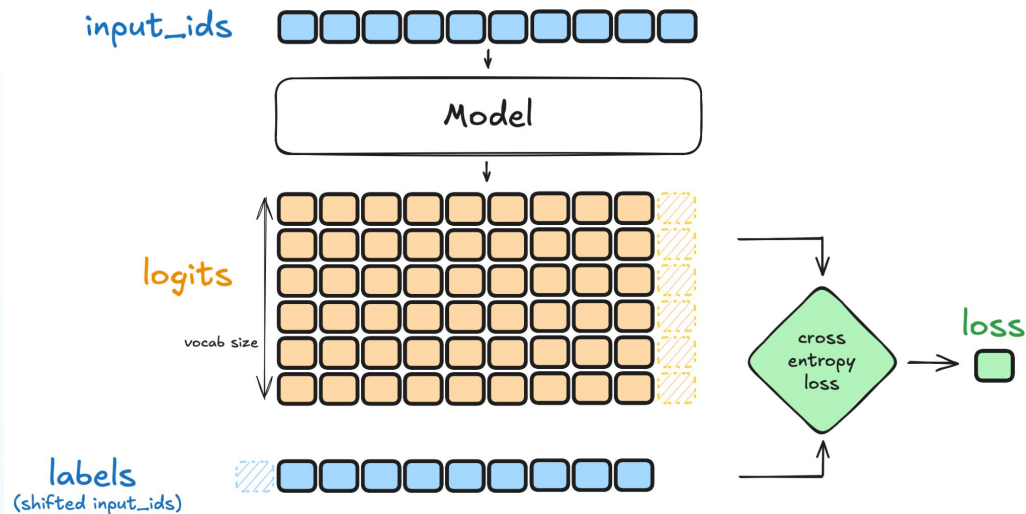
Supports: accelerate, PEFT (LoRA, QLoRA), kernels, vLLM, multimodal...



SFT: imitate a good coding agent

Supervised Fine-Tuning (SFT) teaches the model to **imitate** how a good coding agent works, from its **traces**. Under the hood:

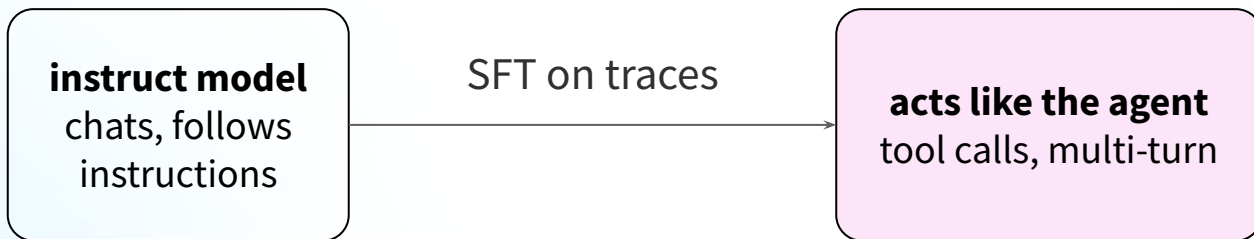
- Goal: minimize negative log-likelihood of a sequence (cross-entropy)
- Simple, stable, and super effective



Why train on traces?

Goal: a small, open model that speaks the agent's language

- It already **chats and follows instructions**, but doesn't know **this** agent's tools or loop
- Traces teach the **tool calls, conventions, and the multi-turn loop**
- The **data is the lesson**, not the hyperparameters



Train only on assistant / tool-call tokens → completion masking.
The model imitates what the agent said, not the tool outputs

Agent traces = conversational format

A trace = messages + tool calls. The model imitates a good agent

[badlogicgames/pi-mono](https://badlogicgames.github.io/pi-mono/) → real pi agent sessions on a real repo

```
# an agent trace: messages + tool calls
messages = [
    {"role": "user", "content": "Fix the failing test in utils.py"},
    {"role": "assistant", "tool_calls": [
        {"name": "read_file", "arguments": {"path": "utils.py"}}]},
    {"role": "tool", "content": "def add(a, b): return a - b"},
    {"role": "assistant", "tool_calls": [
        {"name": "edit", "arguments": {"path": "utils.py", "new": "return a + b"}}]},
] # SFTTrainer applies the chat template automatically
```

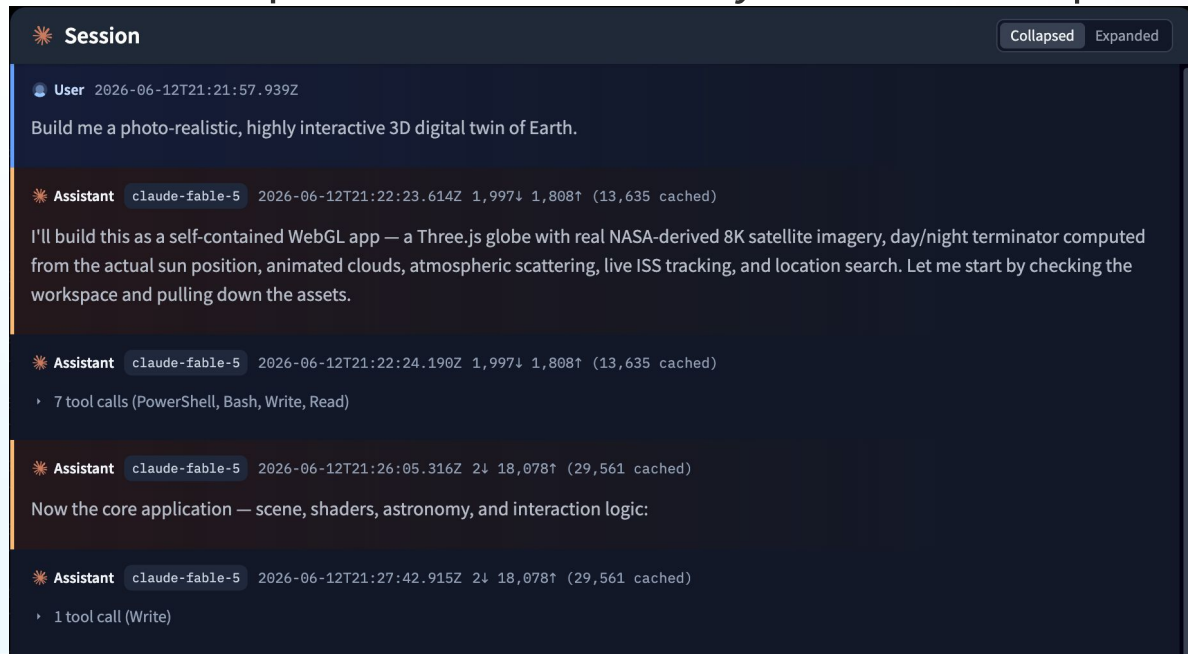
Share them in the open: Hub Agent Traces

We can now share agentic traces on the Hub easily

Supports: Claude Code, Codex & Pi with a dedicated trace viewer

[Example trace](#)

Export and redact with `pi-share-hf` → anonymized before upload



The screenshot displays the Hub Agent Traces viewer interface. At the top, there's a "Session" header with a sun icon and a toggle switch set to "Collapsed". The main content area shows a conversation log with a dark background and light text. The first entry is a "User" message from 2026-06-12T21:21:57.939Z asking to build a photo-realistic 3D digital twin of Earth. This is followed by three "Assistant" messages from the model "claude-fable-5". Each assistant message includes a timestamp, a token count (e.g., 1,997↓ 1,808↑ (13,635 cached)), and a description of the work being done. The first assistant response mentions building a WebGL app with Three.js and NASA imagery. The second response mentions tool calls (PowerShell, Bash, Write, Read). The third response mentions the core application logic (scene, shaders, astronomy, and interaction logic). The interface is clean and modern, with a focus on readability and ease of use.

Session Collapsed Expanded

User 2026-06-12T21:21:57.939Z
Build me a photo-realistic, highly interactive 3D digital twin of Earth.

Assistant `claude-fable-5` 2026-06-12T21:22:23.614Z 1,997↓ 1,808↑ (13,635 cached)
I'll build this as a self-contained WebGL app — a Three.js globe with real NASA-derived 8K satellite imagery, day/night terminator computed from the actual sun position, animated clouds, atmospheric scattering, live ISS tracking, and location search. Let me start by checking the workspace and pulling down the assets.

Assistant `claude-fable-5` 2026-06-12T21:22:24.190Z 1,997↓ 1,808↑ (13,635 cached)
▸ 7 tool calls (PowerShell, Bash, Write, Read)

Assistant `claude-fable-5` 2026-06-12T21:26:05.316Z 2↓ 18,078↑ (29,561 cached)
Now the core application — scene, shaders, astronomy, and interaction logic:

Assistant `claude-fable-5` 2026-06-12T21:27:42.915Z 2↓ 18,078↑ (29,561 cached)
▸ 1 tool call (Write)

SFT is easy to instantiate and train



Only needs a model (even just the name!) and a dataset from the Hub to train
Take advantage of hf jobs

```
from trl import SFTTrainer
from datasets import load_dataset

trainer = SFTTrainer(
    model="Qwen/Qwen3-0.6B",
    train_dataset=load_dataset("badlogicgames/pi-mono", split="train"), # agent traces
)
trainer.train()
```


But imitating $\neq$ knowing how to act

SFT copies good trajectories. That has limits:

- **Outcome:** it doesn't learn from the outcome, only that it “looks like” the traces
- **Errors:** it can't recover from its own mistakes, good traces never show them
- **Ceiling:** the ceiling is the demonstrator

We need the model to learn from its own experience → GRPO

Group Relative Policy Optimization

GRPO was described in [DeepSeekMath](#) paper

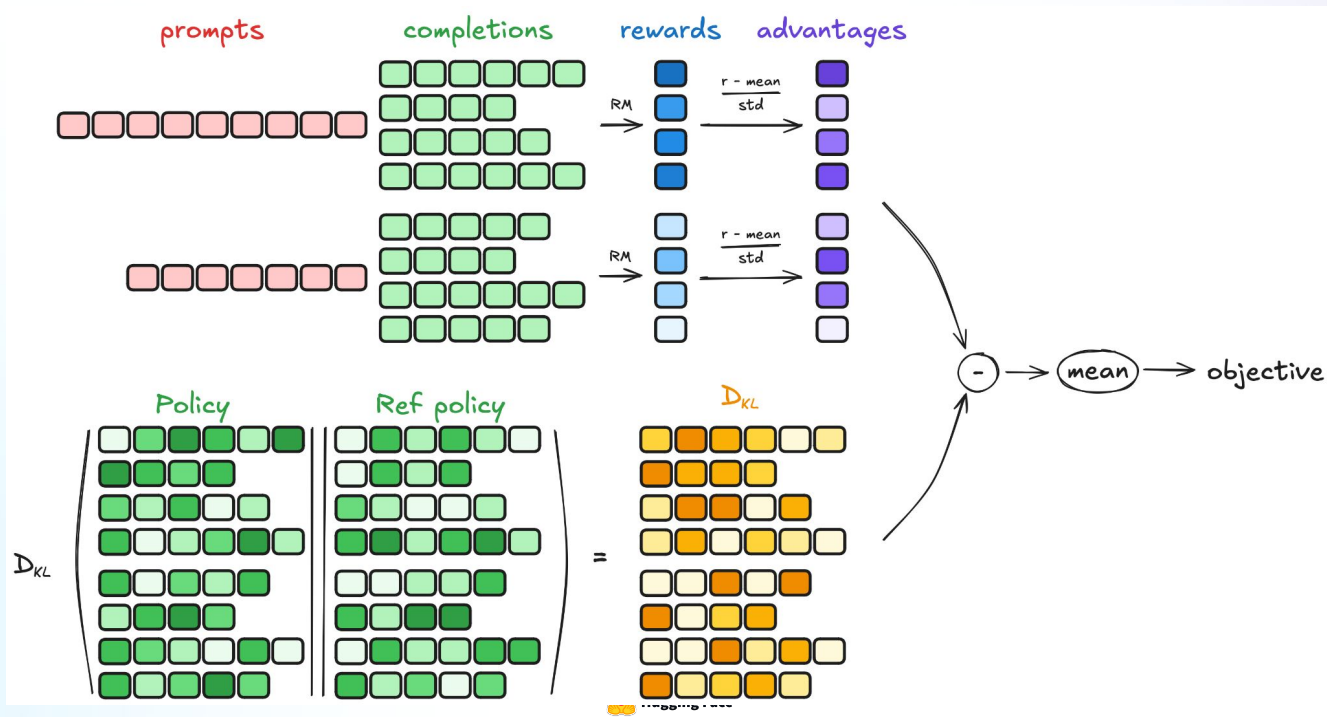
SFT copied the trace (imitate) → GRPO lets the model try the task itself and learn from the result (**reward functions**)

For a coding agent: model writes the solution, we **run the tests**, reward = tests pass. Attempts that pass get reinforced (RLVR)

Rewarding the outcome is what makes the model learn to **reason** its way there

GRPO

Steps: generating completions (online method), computing advantage, computing the loss



GRPOTrainer and GRPOConfig

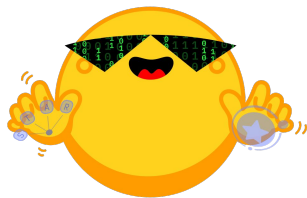
```
from trl import GRPOTrainer, GRPOConfig # AsyncGRPOTrainer for harness/multi-turn

def tests_pass_reward(completions, **kwargs):
    # RLVR: verifiable reward, do the tests pass?
    return [1.0 if run_tests(c) else 0.0 for c in completions]

trainer = GRPOTrainer(
    model="Qwen/Qwen3-0.6B",
    reward_funcs=[tests_pass_reward],
    train_dataset=..., # coding tasks
)
trainer.train() # attempts that pass the tests get reinforced
```

To learn by acting, you need an environment

- So far: static datasets + a reward on a single completion
- A coding agent acts **many turns**: read → edit → run tests → fix → ...
- Every environment has its own API, format logic → **no standard?**



OpenEnv: open environments for agentic RL

- **Gymnasium-style interface:** `reset()` / `step()`; public methods → tools (via MCP)
- **Many environments:** `coding_env`, `opencode_env`, `jupyter`, `repl`, `browser`, `games`, `CARLA` simulator...
- **Runs local, Docker, or cloud** (HF Spaces, HF Sandbox)
- **Native integration with TRL's GRPOTrainer / AsyncGRPOTrainer**
- **White-box or black-box:** trainer owns sampling (train) or reads outputs (eval)

The environment can be anything, but today we focus on coding

OpenEnv

The open source community
for agentic RL



Hugging Face



PyTorch



Reflection



unsloth



NVIDIA



Microsoft



PRIME Intellect



Modal



MERCOR



fleet



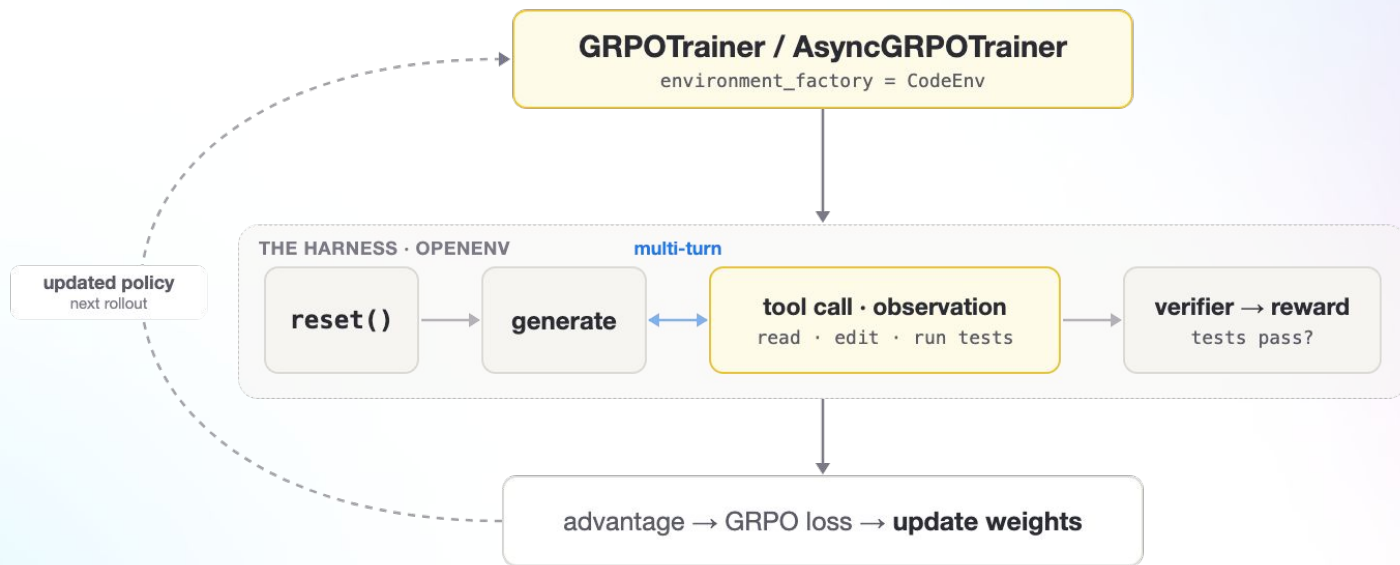
RadixArk

<https://huggingface.co/blog/openenv-agentic-rl>



Hugging Face

Harness-driven training



The model trains inside the real harness (white-box) it acts in
Multi-turn rollouts are long, so this runs async: **AsyncGRPOTrainer** decouples generation from training

Being built in the open, right now

OpenEnv already ships the coding envs.

Currently, the agentic-harness layer and TRL connection is landing: driving multi-turn rollouts (reset / step + a verifier) for training

- **The coding harness (OpenEnv):** multi-turn coding-agent rollouts → act in the sandbox, read the verifier's reward. Developed in the open.
- **Train runs async (AsyncGRPOTrainer in TRL):** decouples generation from training → really long multi-turn async rollouts + weight transfer

Scale it up, or train something tiny

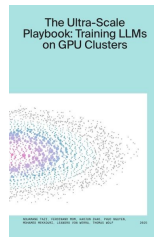
- Train a tiny model:

- LoRA, QLoRA
- Liger kernels 
- Free Colab [examples](#)
- Unsloth 



- Scale the whole training (*1T-param* models):

- AsyncGRPOTrainer (async rollouts)
- HF sandboxes
- Agentic RL is also an infra/systems [problem](#)



Resources



- Concepts:
 - [Agent glossary](#)
- Tools
 - [TRL](#)
 - [OpenEnv](#)
- Data & Traces
 - [Pi traces](#)
 - [Hub Agent Traces](#)
- Recipes/Examples
 - [Scripts](#)
 - [Notebooks](#)
 - [Cookbooks](#)
 - [Example guides](#)
 - [Blogs](#)
 - [Courses](#)
- Post-training & RL envs blogs:
 - [Collection](#)



Thank you

Sergio Paniego Blanco (@[sergiopaniego](https://twitter.com/sergiopaniego))
Machine Learning Engineer @ Hugging Face

